Design and Verification of a 5-Stage Pipelined RV32I Processor with Dynamic Hazard Resolution

Dhruv Deshpande
Student
https://github.com/dhruv-deshpande
dhruvd2405@gmail.com

Abstract - This paper presents the design, implementation, and verification of a 32-bit, 5-stage pipelined processor based on the RISC-V (RV32I) instruction set architecture. While pipelining significantly increases overall instruction throughput compared to single-cycle designs, the overlapping execution of instructions introduces inherent structural and data hazards. To maintain an optimal cycles-per-instruction (CPI) ratio, this architecture integrates a Harvard-style split memory system to prevent structural bottlenecks, alongside a dedicated Hardware Hazard Unit to dynamically resolve Read-After-Write (RAW) dependencies. By utilizing internal data forwarding (bypassing), the processor actively intercepts and routes computed execution data directly back to the arithmetic logic unit (ALU), effectively eliminating the need for pipeline stalls or software-inserted NOPs. The complete top-level datapath was rigorously verified using SystemVerilog testbenches and GTKWave simulations, successfully demonstrating seamless instruction execution, dynamic branch target calculation, and real-time hazard resolution.

Index Terms - RISC-V, Pipelined Architecture, SystemVerilog, Data Hazards, Forwarding Unit.

INTRODUCTION

RISC-V is an open-source instruction set architecture used to develop custom processors for a variety of applications, from embedded designs to supercomputers. Originally developed at the University of California, Berkeley, the RISC-V ISA is considered the fifth generation of processors built on the concept of the reduced instruction set computer (RISC). Essentially, the RISC-V architecture allows designers to customize and build their processors in a way that's tailored to their target end applications, so they can optimize the power, performance, and area (PPA) for those applications. The RISC-V ISA also provides the flexibility to pick and choose from available features, rather than having to use the full feature set.

While single-cycle processor designs offer functional simplicity by executing one complete instruction per clock cycle, they are fundamentally limited by their critical path length, which dictates a slow clock frequency. To achieve higher instruction throughput, modern microarchitectures rely on pipelining to exploit instruction-level parallelism. By decomposing the datapath into five distinct functional stages, Instruction Fetch (IF), Instruction Decode (ID), Execute (EX), Memory Access (MEM), and Writeback (WB), multiple instructions can be processed concurrently in a cascading waterfall structure. Ideally, a 5-stage pipeline achieves a Cycles Per Instruction (CPI) approaching 1.0.

This paper details the design, implementation, and verification of a 32-bit, 5-stage pipelined RV32I processor core with a focus on dynamic data hazard resolution.

PIPELINING

We design a pipelined processor by subdividing the single-cycle processor into five pipeline stages. Thus, five instructions can execute simultaneously, one in each stage. Because each stage has only one-fifth of the entire logic, the clock frequency is approximately five times faster.

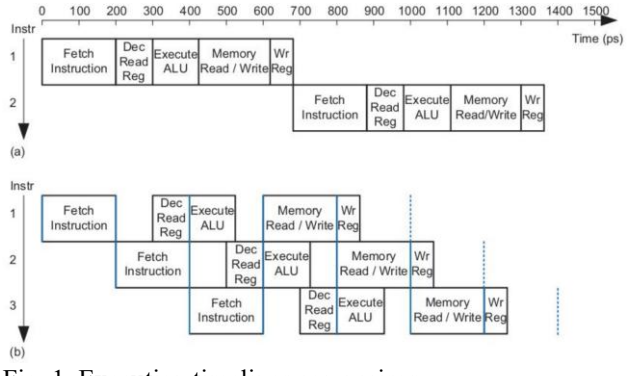

Fig. 1. Execution timelines comparison

Timing Shift: In the single-cycle model, an entire instruction executes in one 800 ps clock cycle. The pipelined model divides this process, standardizing its clock cycle to match the longest individual stage (200 ps).

Latency vs. Throughput: Pipelining introduces a vital trade-off. Instruction latency increases (taking 1000 ps total instead of 800 ps), but overall throughput dramatically improves. Once the pipeline is full, a new instruction finishes every 200 ps, compared to every 800 ps in the single-cycle design.

Real-World Limits: A perfect 5x speedup is purely theoretical. In practice, pipeline hazards (data, structural, and control dependencies) and register delays cause stalls, slightly reducing this ideal efficiency.

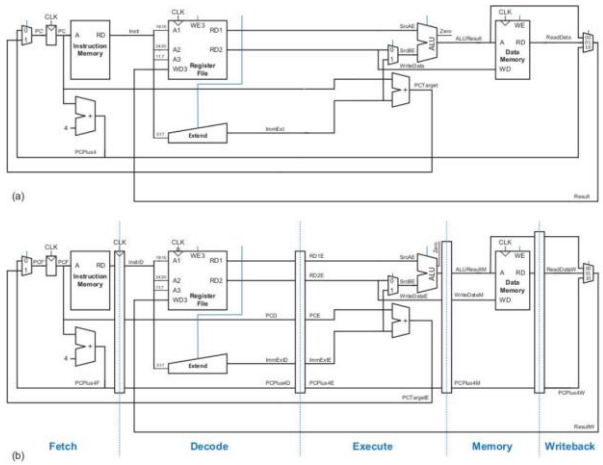

Fig. 2. Processor architecture comparison

This figure illustrates the architectural differences between a single-cycle datapath and its pipelined counterpart. In the single-cycle datapath (a), an instruction flows continuously from the Instruction Memory through the Register File, ALU, and Data Memory. There are no barriers, meaning the system must wait for the signals to propagate through the entire circuit before the clock cycle can end and the next instruction can be fetched.

To achieve pipelined timing, the hardware must physically isolate each stage so multiple instructions can be processed simultaneously without their signals interfering. This is shown in datapath (b) through the addition of pipeline registers (the vertical rectangular bars separating the stages). These registers act as temporary holding areas. At the end of every clock cycle, the results of one stage are saved into the pipeline register and passed to the next stage at the start of the new clock cycle.

By dividing the datapath into Fetch, Decode, Execute, Memory, and Writeback stages, the pipeline registers ensure that ALU, for example, can perform calculations for Instruction 2 at the exact same time the Instruction Memory is fetching Instruction 3. Each stage contains only the specific data and control signals needed for the instruction currently occupying that segment of the hardware.

PIPELINE DATAPATH

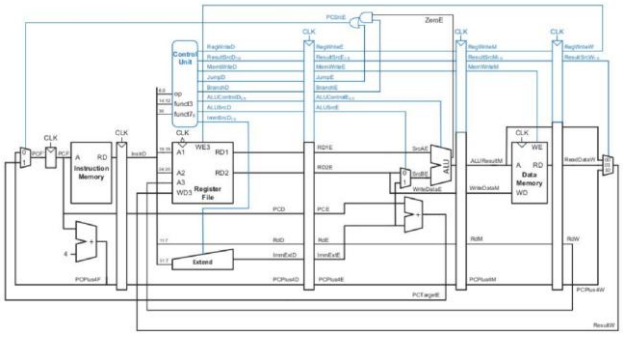
Fig. 3. Complete 5-stage pipeline datapath

This figure introduces the Control Unit into the pipelined architecture. In a single-cycle datapath, the control unit reads the instruction opcode and immediately broadcasts all necessary control signals (such as memory read/write enables or ALU operation codes) to the entire processor at once. However, in a pipelined datapath, this immediate broadcast would cause chaos. If the control unit generates a "Write to Memory" signal during the Decode stage, that signal cannot be sent to the Data Memory immediately, because the Data Memory is currently working on an entirely different instruction that is further ahead in the pipeline.

To solve this, the control signals themselves must be pipelined. As shown by the blue signal lines in the diagram, the control signals are generated during the Decode stage and are then passed into the pipeline registers alongside the data. These signals travel with their specific instruction from stage to stage. When the instruction moves from Execute to Memory, its specific memory control signals are finally released from the pipeline register to activate the Data Memory. This ensures that the control signals stay perfectly synchronized with the instructions they belong to as it flows through the hardware.

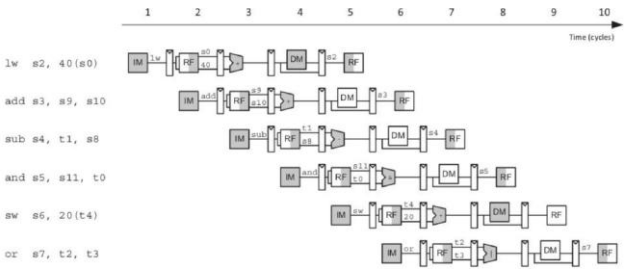

Fig. 4. Abstract view of pipelining

This space-time diagram illustrates the ideal, uninterrupted flow of multiple instructions through the 5-stage pipeline over a period of 10 clock cycles. By tracing the execution vertically down a specific clock cycle, we can observe concurrent execution in action. For example, during Cycle 5, the pipeline achieves maximum utilization: it is fetching the sw instruction, decoding the and instruction, executing the sub instruction, accessing memory for the add instruction, and writing back the result of the initial lw instruction simultaneously.

The diagram also visually confirms the throughput benefits of pipelining. While the very first instruction (lw) requires five full cycles to complete, every subsequent instruction finishes just one cycle later. From Cycle 5 onward, one instruction completes and exits the pipeline on every single clock tick, achieving the theoretical ideal of one instruction per cycle.

Critical hardware optimization is depicted in the Register File (RF) stages. RF block is shaded on the right half during the Decode stage and shaded on the left half during the Writeback stage. This indicates a "split-cycle" operation: the processor writes data to the registers during the first half of the clock cycle and reads from the registers during the second half. This resolves potential data hazards, allowing an instruction to read a register in the exact same cycle that an older instruction is writing to it.

Instruction Formats

The RISC-V ISA utilizes a 32-bit fixed-length instruction word categorized into six primary formats. To minimize decoding complexity, the source registers (rs1, rs2) and destination register (rd) remain in identical physical bit positions across all formats.

The six instruction formats are described below:

1) R-Type (Register): Utilized for standard arithmetic and logical operations where all operands reside in registers. Layout: funct7, rs2, rs1, funct3, rd [11:7], opcode [6:0].

2) I-Type (Immediate): Utilized for arithmetic with short constants and memory loads. The rs2 field is replaced by a 12-bit immediate payload. Layout: imm[11:0], rs1, funct3, rd [11:7], opcode [6:0].

3) S-Type (Store): Utilized exclusively for storing data to main memory. The 12-bit immediate is split into two fields to preserve the standard physical locations of the source registers. Layout: imm[11:5], rs2, rs1, funct3, imm[4:0] [11:7], opcode [6:0].

4) B-Type (Branch): Utilized for conditional control flow. Structurally similar to S-Type, but the immediate bits are scrambled. The 0th bit is hardwired to zero, allowing the 12-bit field to represent a 13-bit offset. Layout: imm[12|10:5], rs2, rs1, funct3, imm[4:1|11] [11:7], opcode [6:0].

5) U-Type (Upper Immediate): Utilized for loading large 20-bit constants directly into the upper bits of a destination register. Layout: imm[31:12], rd [11:7], opcode [6:0].

6) J-Type (Jump): Utilized for unconditional target jumps. Dedicates 20 bits to a scrambled immediate payload for large memory offsets. Layout: imm[20|10:1|11|19:12], rd [11:7], opcode [6:0].

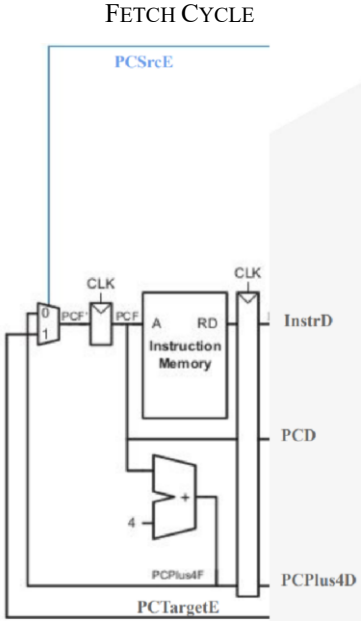

Fig. 5. Schematic of the Fetch stage

The first stage of the pipelined datapath is responsible for retrieving the next instruction to be executed. The operation centers around the Program Counter (PC), a register that holds the memory address of the current instruction. (Note: The instant the clock transitions from 0 to 1 (the rising edge), the PC snaps up whatever new value is waiting at its input, memorizes it, and outputs it. It holds that new value until the next rising edge). During a single clock cycle, the PC address is routed to the Instruction Memory, which reads the corresponding memory block and outputs the 32-bit instruction (InstrD).

Simultaneously, the datapath must calculate the address for the next instruction. The PC value is routed to a dedicated Adder, which increments the address by 4 bytes (standard for a 32-bit architecture) to produce PCPlus4F. The PC Mux at the far left determines whether the PC will update to this sequential address (PCPlus4F) or jump to a new branch address (PCTargetE) calculated in a later execution stage. The multiplexer's choice is dictated by the control signal PCSrcE.

At the end of the clock cycle, the system reaches the pipeline boundary. The fetched instruction, the current PC address (PCD), and the incremented PC address (PCPlus4D) are all locked into the Fetch Stage Registers (often called the IF/ID pipeline register). This securely buffers the data so the Fetch stage can immediately begin retrieving the next instruction on the next clock cycle, while the rest of the processor works on the current one.

Concept of Byte-Addressable Memory (PCPlus4F): In this architecture, every single instruction is exactly 32 bits long. However, memory is organized into bytes, and 1 byte = 8 bits. Because an instruction is 32 bits long, it requires 4 bytes of space to store a single instruction (32 ÷ 8 = 4). Instruction 1 lives in boxes 0, 1, 2, and 3. Its official address is 0. Instruction 2 lives in boxes 4, 5, 6, and 7. Its official address is 4. So, the Adder automatically adds 4 to the PC to point exactly to the start of the next sequential instruction.

```
PS D:\Coursera\RISC-V> iverilog -g2012 -o sim.out decode_cycle.v decode_cycle_tb.sv components2.v
>> vvp sim.out
Time: 0 | rst: 0 | InstrD: 00000000 | PCE: 00000000 | RD_E:  0 | RS1_E:  0
Time: 20 | rst: 1 | InstrD: 00000000 | PCE: 00000000 | RD_E:  0 | RS1_E:  0
Time: 30 | rst: 1 | InstrD: 002081b3 | PCE: 00000000 | RD_E:  0 | RS1_E:  0
Time: 35 | rst: 1 | InstrD: 002081b3 | PCE: 00000010 | RD_E:  3 | RS1_E:  1
Time: 40 | rst: 1 | InstrD: 00508213 | PCE: 00000010 | RD_E:  3 | RS1_E:  1
Time: 45 | rst: 1 | InstrD: 00508213 | PCE: 00000014 | RD_E:  4 | RS1_E:  1
decode_cycle_tb.sv:86: $finish called at 90 (1s)
PS D:\Coursera\RISC-V>
```

Fig. 6. Fetch Cycle terminal output from the testbench

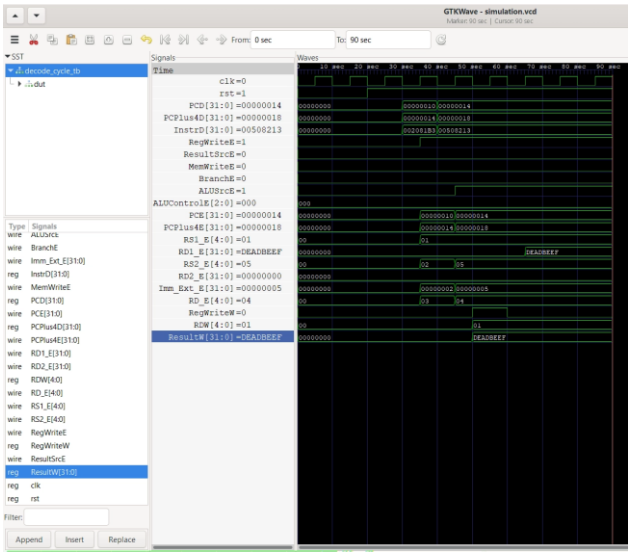

Fig. 7. Fetch Cycle GTKWave simulation waveform

To validate the architecture of the Fetch stage, a SystemVerilog testbench was simulated using Icarus Verilog, with the resulting signal transitions analyzed in GTKWave (Fig. 6 and Fig. 7). The simulation systematically verifies the system reset protocol, standard sequential instruction fetching, and dynamic branch target resolution.

At simulation start (t = 0), the uninitialized pipeline registers exhibit undefined states (XXXXXXXX). At t = 2, the active-low reset (rst) is asserted. The waveform demonstrates the asynchronous nature of the reset block; independent of the clock edge, the PCD, PCPlus4D, and InstrD output lines are instantly forced to 0x00000000, ensuring a safe known state before execution begins.

Following the release of the reset signal at t = 17, the datapath enters standard execution. The hardware successfully increments the Program Counter by four bytes per clock cycle (0x00000000 → 0x00000004 → 0x00000008). Concurrently, the Instruction Memory successfully serves the corresponding 32-bit machine code (01100013, 02200023, 03300033). The GTKWave timing diagram visually confirms the required 1-cycle pipeline delay: the fetched instruction and incremented PC appear at the InstrD and PCD pipeline boundaries exactly one rising clock edge after the internal PC register updates.

To verify multiplexer control logic, a branch scenario is injected at t = 57. The PCSrcE select line is driven high, and the branch target address 0x0000A000 is applied. The simulation confirms that the hardware instantly overrides the sequential adder. At t = 67, the PCD successfully jumps to 0x0000A000, and the memory correctly fetches the subsequent target instruction (0xBEEFCAFE). Upon clearing the branch signal, the processor seamlessly resumes sequential fetching from the new base address.

DECODE CYCLE

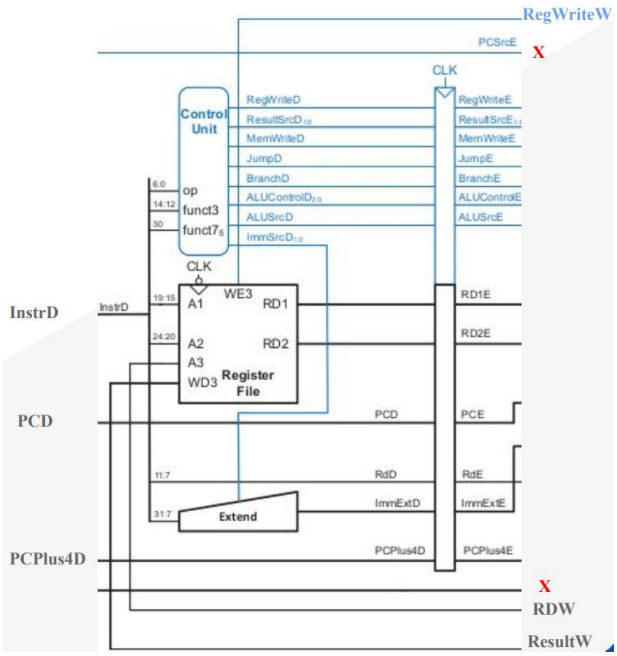

Fig. 8. Schematic of the Decode stage

Fig. 8 illustrates the hardware schematic for the Instruction Decode stage of the processor pipeline. This stage is responsible for parsing the 32-bit fetched instruction (InstrD), retrieving operand data, generating datapath control signals, and latching the results for the Execute stage. The architecture consists of three primary functional blocks: the Control Unit, the Register File, and the Sign Extension Unit.

Upon entering the Decode stage, the 32-bit instruction is immediately sliced into specific fields. The opcode (bits 6:0), funct3 (bits 14:12), and funct7 (bit 30) fields are routed directly into the Control Unit. Based on these fields, the Control Unit acts as the brain of the instruction, decoding the operation and asserting the necessary control signals (RegWriteD, ALUSrcD, MemWriteD, etc.) that will dictate the behavior of the downstream Execute, Memory, and Writeback stages.

Simultaneously, the source register addresses are extracted from the instruction and fed into the Register File. Bits drive the first read address (A1) to fetch source register 1 (RD1), while bits drive the second read address (A2) to fetch source register 2 (RD2). The destination register address (RdD), located at bits [11:7], bypasses the register file entirely and is forwarded down the pipeline to be used later during the Writeback phase. Notably, the diagram indicates feedback lines from the Writeback (W) stage (RegWriteW, RDW, ResultW), which act as the write-enable, write-address, and write-data inputs (A3, WD3, WE3) to asynchronously update the Register File when an instruction completes.

For instructions that utilize immediate values (such as I-type or S-type instructions), the upper payload of the instruction (bits 31:7) is routed to the Extend unit (This unit takes that small number and pads it with extra zeros (or ones, if it is a negative number) until it is exactly 32 bits wide, so it fits perfectly into the math unit later). The Control Unit's ImmSrcD signal dictates how this hardware extracts and sign-extends the specific immediate bits into a standardized 32-bit format (ImmExtD), ensuring mathematical consistency for the ALU.

At the far right of the schematic, all decoded data and control signals converge at the ID/EX pipeline register. On the rising edge of the system clock (CLK), this synchronous barrier latches the read data (RD1, RD2), the extended immediate (ImmExtD), the forwarded PC values (PCD, PCPlus4D), and all Control Unit flags. This effectively translates the combinatorial 'D' signals into stable 'E' signals, securely passing the workload to the Execute stage for the next clock cycle.

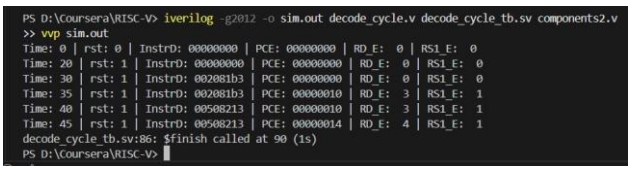

Fig. 9. Decode Cycle terminal output from the testbench

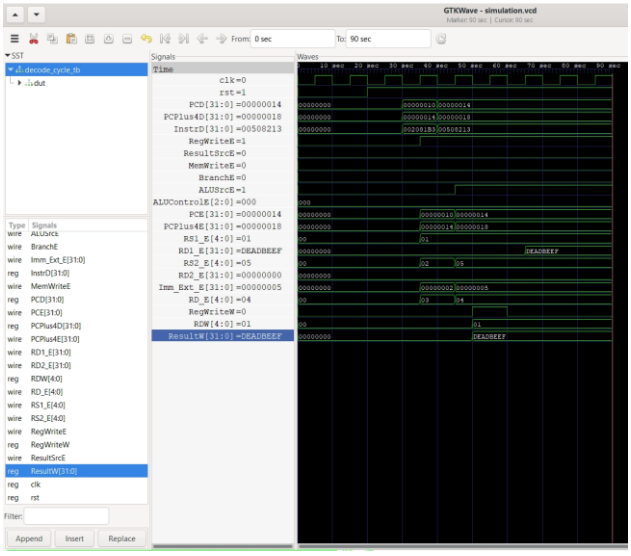

Fig. 10. Decode Cycle GTKWave simulation waveform

To validate the architecture of the Decode stage, a SystemVerilog testbench was simulated using Icarus Verilog, with the resulting signal transitions analyzed in GTKWave (Fig. 9 and Fig. 10). The simulation systematically verifies the system reset protocol,

combinatorial instruction decoding, synchronous pipeline latching, and the register file writeback mechanism.

At simulation start (t = 0), the active-low reset (rst) is asserted (rst = 0). The terminal output and waveform demonstrate the asynchronous reset behavior; independent of active clocking, the internal registers and ID/EX pipeline outputs, such as PCE, RD_E, and RS1_E are instantly forced to zero. This ensures a safe, known state and prevents the propagation of undefined data into the execution phase before standard operation begins.

Following the release of the reset signal at t = 20, the datapath enters standard active decoding. The GTKWave timing diagram and terminal log visually confirm the required 1-cycle pipeline delay inherent to the ID/EX boundary. For instance, when the machine code 0x002081b3 is presented at the InstrD port at t = 30, the combinatorial logic immediately extracts the operand addresses. However, the corresponding execution stage signals (RD_E = 3, RS1_E = 1) do not update until the subsequent clock transition at t = 35. Similarly, when an I-type instruction (0x00508213) is fetched at t = 40, the sign-extension unit correctly updates the Imm_Ext_E bus to 0x00000005 exactly one clock edge later at t = 45.

To verify the Register File's feedback loop, a writeback scenario is injected into the simulation. The testbench drives the RegWriteW signal high and applies the known data word 0xDEADBEEF to the ResultW bus, alongside a destination address of RDW = 0x01. The waveform confirms that this data is successfully committed to the internal register file. When the source address RS1_E subsequently specifies register 0x01, the read bus RD1_E accurately outputs 0xDEADBEEF. This verifies that the decode stage effectively processes backward-propagating data from the WB stage, a critical function for resolving data hazards.

EXECUTE CYCLE

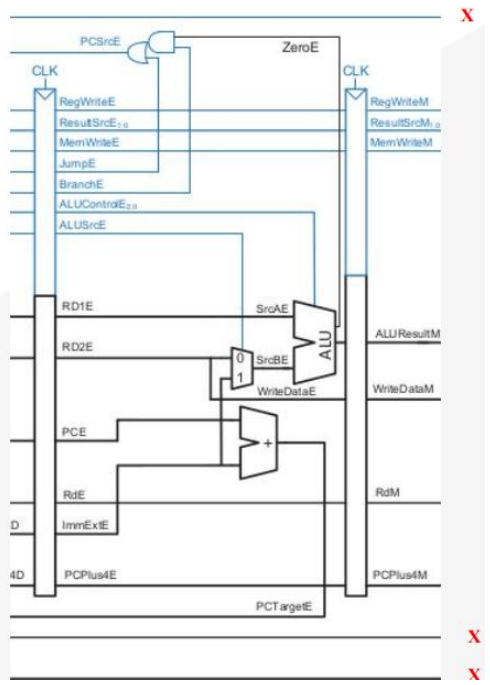

Fig. 11. Schematic of the Execute stage

Fig. 11 illustrates the hardware schematic for the Execute (EX) stage of the processor pipeline. This stage is responsible for performing arithmetic and logical computations, calculating memory addresses, and evaluating control flow conditions.

Upon entering the Execute stage, data and control signals from the ID/EX pipeline registers are routed to their respective functional blocks. The central computational component is the Arithmetic Logic Unit (ALU). The first ALU operand (SrcAE) is sourced directly from the first register read data bus (RD1E). The second ALU operand (SrcBE) is determined by a multiplexer governed by the ALUSrcE control signal. Depending on the specific instruction class, this multiplexer selects either the second register read data (RD2E) for standard R-type operations, or the sign-extended immediate value (ImmExtE) for I-type arithmetic and memory access instructions.

Concurrently, the datapath resolves potential control flow diversions. A dedicated address adder calculates the branch target address (PCTargetE) by summing the current program counter (PCE) and the sign-extended immediate offset (ImmExtE). Simultaneously, the ALU evaluates the arithmetic condition of the operation, outputting a ZeroE flag. The pipeline's branch resolution logic utilizes an AND-OR gate configuration to evaluate these parameters: if a branch instruction is active (BranchE) and the arithmetic condition is met (ZeroE), or if an unconditional jump instruction is active (JumpE), the select signal

PCSrcE is asserted. This signal is routed back to the Fetch stage to override the sequential program counter with the calculated target address.

At the rightmost boundary, the combinatorial outputs are secured for the subsequent memory phase. On the rising edge of the system clock (CLK), the computed ALU result (ALUResultM), the data intended for memory storage (WriteDataM, sourced directly from RD2E), the destination register address (RdM), the incremented program counter (PCPlus4M), and all surviving control signals are synchronously latched into the EX/MEM pipeline registers.

```
PS D:\Coursera\RISC-V> iverilog -g2012 -o sim.out execute_cycle.v execute_cycle_tb.sv components3.v
>> vvp sim.out
Time=0 | rst=0 | FwdA=00 FwdB=00 ALUSrc=0 | PCSrcE=0 PCTargetE=00000000 | ALU_ResultM=0 (Pipelined)

--- Test Case 1: Standard ALU Operation (No Forwarding) ---
Time=15 | rst=1 | FwdA=00 FwdB=00 ALUSrc=0 | PCSrcE=0 PCTargetE=00000000 | ALU_ResultM=0 (Pipelined)
Time=25 | rst=1 | FwdA=00 FwdB=00 ALUSrc=0 | PCSrcE=0 PCTargetE=00000000 | ALU_ResultM=40 (Pipelined)

--- Test Case 2: Forwarding from Writeback & Immediate Operand ---
Time=26 | rst=1 | FwdA=01 FwdB=00 ALUSrc=1 | PCSrcE=0 PCTargetE=00000032 | ALU_ResultM=40 (Pipelined)

--- Test Case 3: Branch Evaluation (Branch Taken) ---
Time=35 | rst=1 | FwdA=00 FwdB=00 ALUSrc=0 | PCSrcE=1 PCTargetE=00000110 | ALU_ResultM=150 (Pipelined)
Time=45 | rst=1 | FwdA=00 FwdB=00 ALUSrc=0 | PCSrcE=0 PCTargetE=00000110 | ALU_ResultM=0 (Pipelined)

Simulation Complete.
execute_cycle_tb.sv:114: $finish called at 65 (1s)
PS D:\Coursera\RISC-V>
```

Fig. 12. Execute Cycle terminal output from the testbench

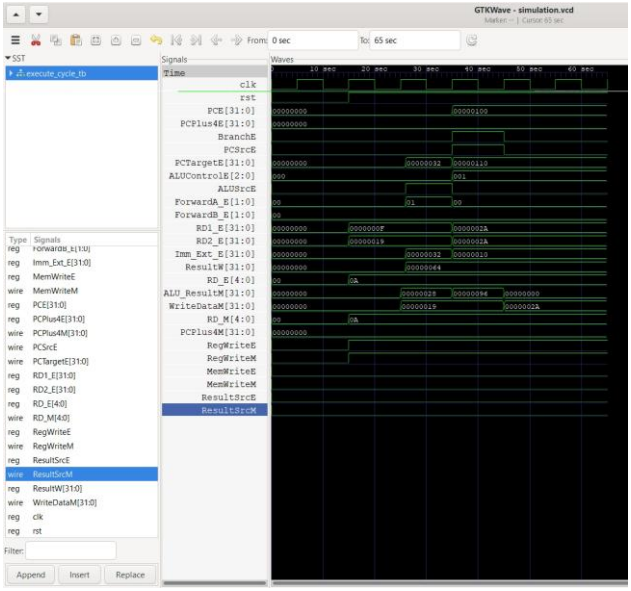

Fig. 13. Execute Cycle GTKWave simulation waveform

To validate the architecture of the Execute stage, a SystemVerilog testbench was simulated using Icarus Verilog, with the resulting signal transitions analyzed in GTKWave (Fig. 12 and Fig. 13). The simulation systematically verifies the computational integrity of the Arithmetic Logic Unit (ALU), the dynamic resolution of data hazards via the Forwarding Unit, and the calculation and evaluation of control flow branches.

At simulation start (t = 0), the active-low reset is asserted (rst = 0). The terminal log confirms that all execution and memory boundary signals, including the branch control line (PCSrcE), the target address bus (PCTargetE), and the pipelined ALU result (ALU_ResultM)—are instantly forced to zero, ensuring a safe, known state before active execution commences.

Following the release of the reset signal, the datapath undergoes standard execution (Test Case 1). The waveform visually confirms the inherent 1-cycle pipeline delay; operands computed in the EX stage are successfully latched into the EX/MEM pipeline registers, appearing on the ALU_ResultM bus on the subsequent clock edge.

A critical feature of the advanced pipelined architecture—dynamic data forwarding—is validated in Test Case 2. At t = 26, the terminal log indicates that the primary operand multiplexer control signal is driven to 01 (FwdA = 01). This confirms that the hardware successfully detects a Read-After-Write (RAW) data hazard and dynamically bypasses stale register data, instead routing a forwarded result directly from the Writeback stage into the ALU. Concurrently, the ALUSrc control signal asserts high (ALUSrc = 1), demonstrating the ALU's capability to process sign-extended immediate operands in tandem with forwarded data.

Finally, Test Case 3 verifies the branch resolution logic. At t = 35, a branch condition is evaluated as true. The datapath successfully computes the branch target address by adding the immediate offset to the current program counter, yielding a target of 0x00000110 on the PCTargetE bus. Simultaneously, the combinatorial logic correctly asserts the branch select signal (PCSrcE = 1). As demonstrated in the waveform, this signal cleanly propagates out of the Execute stage to command the Fetch stage multiplexers to jump to the newly calculated target address, successfully altering the processor's control flow.

MEMORY CYCLE

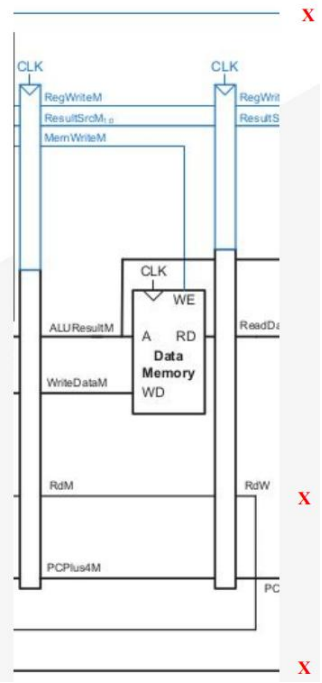

Fig. 14. Schematic of the Memory stage

Fig. 14 illustrates the hardware schematic for the Memory (MEM) stage of the processor pipeline. This stage governs all interactions with the Data Memory, handling both load and store operations based on the effective addresses calculated in the preceding Execute stage.

Upon entering the Memory stage, the synchronous EX/MEM pipeline registers provide the necessary data and control signals. For memory access instructions, the computed ALU result (ALUResultM) is routed directly to the address port (A) of the Data Memory block. In the event of a store operation, the MemWriteM control signal is asserted, acting as the active-high write-enable (WE) flag. Synchronized with the system clock (CLK), this action commits the payload residing on the WriteDataM bus into the specified physical memory location.

Conversely, during a load operation, the Data Memory retrieves the contents located at the designated ALUResultM address and outputs the payload via the read data (RD) port. For standard arithmetic, logical, or control flow instructions that do not require memory interaction, the memory block is functionally bypassed.

At the rightmost boundary, the datapath reaches the final pipeline barrier. On the rising edge of the system clock, the retrieved memory data (ReadData), the bypassed ALU result, the destination register address (RdM transitioning to RdW), the incremented program counter (PCPlus4M), and the surviving writeback control signals (RegWriteM, ResultSrcM) are synchronously latched into the MEM/WB pipeline registers. This synchronous buffering isolates the Memory stage outputs, presenting a stable state for the final Writeback phase in the subsequent clock cycle.

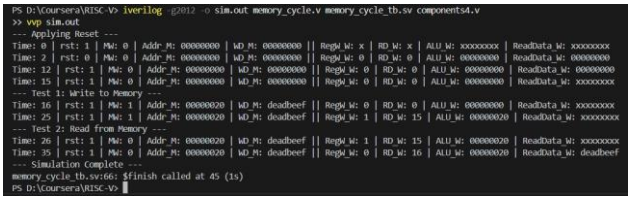

Fig. 15. Memory Cycle terminal output from the testbench

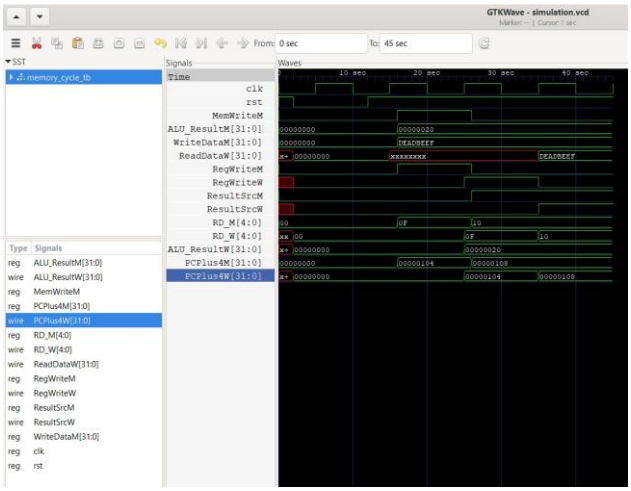

Fig. 16. Memory Cycle GTKWave simulation waveform

To validate the architecture of the Memory stage, a SystemVerilog testbench was simulated using Icarus Verilog, with the resulting signal transitions analyzed in GTKWave (Fig. 15 and Fig. 16). The simulation systematically verifies the system reset protocol, active memory store operations, and the synchronous retrieval and latching of memory read data across the MEM/WB pipeline boundary.

At the onset of the simulation, the active-low reset signal is briefly asserted (rst = 0 at t = 2). The terminal log indicates that this correctly flushes the pipeline registers, clearing undefined states (x) and forcing the writeback boundary signals, such as RD_W and ALU_W, to a known zero state once the reset is released at t = 12.

Following initialization, Test 1 validates the datapath's capability to execute store instructions. At t = 15, the control logic asserts the memory write-enable signal (MW = 1 / MemWriteM). Simultaneously, the calculated target address 0x00000020 is presented on the ALU_ResultM bus, and the test payload 0xDEADBEEF is driven onto the WriteDataM bus. The waveform confirms that these

conditions successfully commit the payload into the specified physical address within the Data Memory block.

To verify data retrieval and synchronous pipeline buffering, Test 2 simulates a load instruction targeting the previously written address. The write-enable signal is safely deasserted (MW = 0), and the address 0x00000020 is maintained. The memory block successfully accesses the stored payload; however, demonstrating the necessary 1-cycle pipeline delay, the terminal log and waveform confirm that the data does not immediately propagate. It is only upon the subsequent clock edge (t = 35) that the retrieved payload 0xDEADBEEF is securely latched into the MEM/WB pipeline register, appearing stably on the ReadData_W bus. Concurrently, the destination register address (RD_M = 0x10) successfully propagates to RD_W, ensuring all data and control directives are properly synchronized for the impending Writeback stage.

WRITEBACK CYCLE

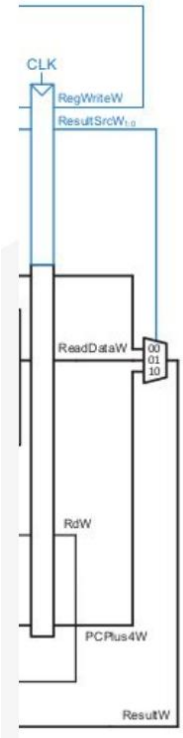

Fig. 17. Schematic of the Writeback stage

Fig. 17 illustrates the hardware schematic for the Writeback (WB) stage, the final operational phase of the processor pipeline. This stage is strictly responsible for resolving the ultimate execution data and committing it back to the architectural Register File in the Decode stage.

Upon entering the Writeback stage, the signals latched within the MEM/WB pipeline registers are routed to a primary 3-to-1 multiplexer. The selection logic of this multiplexer is governed by the 2-bit control signal ResultSrcW. Depending on the specific instruction class executed, the multiplexer routes one of three potential data sources to the final ResultW output bus: the computed arithmetic result from the ALU (port 00), the loaded data retrieved from the Data Memory (ReadDataW, port 01), or the incremented program counter (PCPlus4W, port 10), which is essential for storing return addresses during jump-and-link operations.

Once the final data word (ResultW) is dynamically resolved, it is asynchronously propagated backward through the pipeline datapath to the Instruction Decode stage. The payload is presented to the Register File's dedicated write ports alongside the target destination address (RdW) and the active-high write-enable control signal (RegWriteW). If RegWriteW is asserted by the Control Unit, the processor formally writes the data into the specified register, successfully concluding the lifecycle of the instruction.

Crucially, to maintain optimal pipeline throughput and mitigate structural Read-After-Write (RAW) data hazards, the resolved ResultW bus is simultaneously tapped and routed to the Forwarding Unit located in the Execute stage. This internal bypassing mechanism ensures that subsequent dependent instructions have immediate, real-time access to the computed data prior to its formal commitment in the Register File.

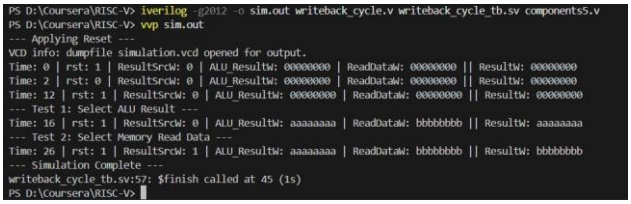

Fig. 18. Writeback Cycle terminal output from testbench

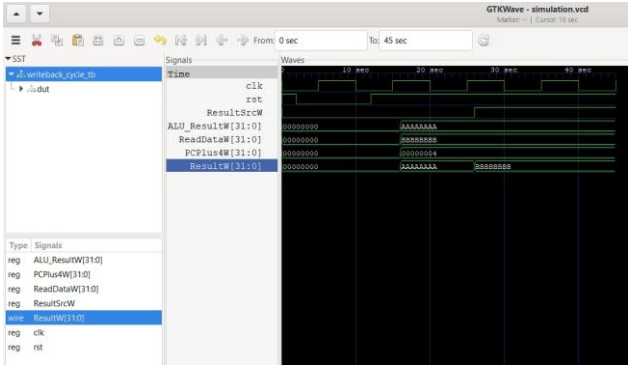

Fig. 19. Writeback Cycle GTKWave simulation waveform

To validate the architecture of the Writeback stage, a SystemVerilog testbench was simulated using Icarus Verilog, with the resulting signal transitions analyzed in

GTKWave (Fig. 18 and Fig. 19). The simulation systematically verifies the system reset protocol and the combinatorial multiplexing logic responsible for resolving the final execution data before it is committed to the Register File.

At the onset of the simulation, the active-low reset signal is asserted (rst = 0 at t = 2). The terminal log and waveform demonstrate that this successfully initializes the datapath, forcing the multiplexer's output bus (ResultW) to a secure zero state (0x00000000). This ensures a safe known state, preventing any uninitialized or spurious data from inadvertently propagating backward into the Decode stage registers. The reset is subsequently released at t = 12.

Following initialization, Test 1 evaluates the datapath's response to a standard arithmetic or logical instruction. At t = 16, the multiplexer control signal ResultSrcW is driven low (0). To test the routing logic, the testbench applies distinct, identifiable payload vectors to the input buses: 0xAAAAAAAA to the ALU_ResultW bus and 0xBBBBBBBB to the ReadDataW bus. The waveform (Fig. 19) and terminal log confirm that the multiplexer correctly isolates the ALU-computed data, presenting 0xAAAAAAAA stably on the final ResultW output bus.

To verify the handling of load instructions, Test 2 simulates a memory retrieval scenario. At t = 26, the ResultSrcW control signal is asserted high (1), instructing the hardware to select the memory data path. The simulation verifies that the combinatorial logic acts instantly; independent of a clock edge, the ResultW bus accurately updates to reflect the memory payload (0xBBBBBBBB). This successfully demonstrates the dynamic resolution of execution data required for diverse instruction classes prior to final register file commitment.

PIPELINE HAZARDS AND DATA DEPENDENCY

While pipelining significantly increases instruction throughput, the overlapping execution of multiple instructions introduces structural and data hazards. A structural hazard occurs when the hardware cannot support the simultaneous execution of overlapping instructions (e.g., a single memory module for both instruction fetch and data access). This architecture entirely avoids structural hazards by utilizing a Harvard-style split memory architecture, isolating the Instruction Memory from the Data Memory.

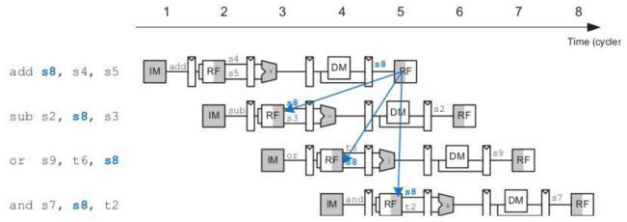

Fig. 20. Data Hazard In Pipelining

However, the architecture must actively manage Data Hazards. Specifically, Read-After-Write (RAW) dependencies occur when an instruction requires an operand that is currently being computed by a preceding instruction and has not yet been committed to the Register File. As illustrated in Fig. 20, if the instruction add s8, s4, s5 computes a new value for register s8, the subsequent instructions (sub, or, and) that rely on s8 will fetch stale data from the Register File if executed in successive clock cycles without intervention.

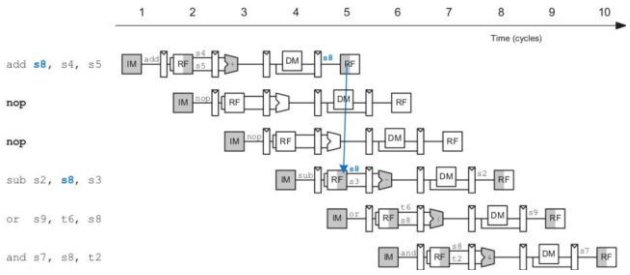

Fig. 21. Using NOPs

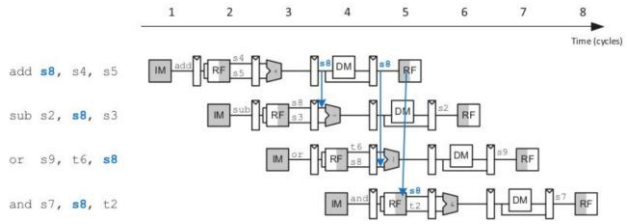

Fig. 22. Using Forwarding / Bypassing

While this hazard can be resolved entirely in software by inserting empty operations (NOPs) into the instruction stream to forcibly stall the pipeline until the data is committed, this severely degrades cycle latency. Therefore, a hardware-based Forwarding (or Bypassing) technique is implemented to maintain optimal throughput.

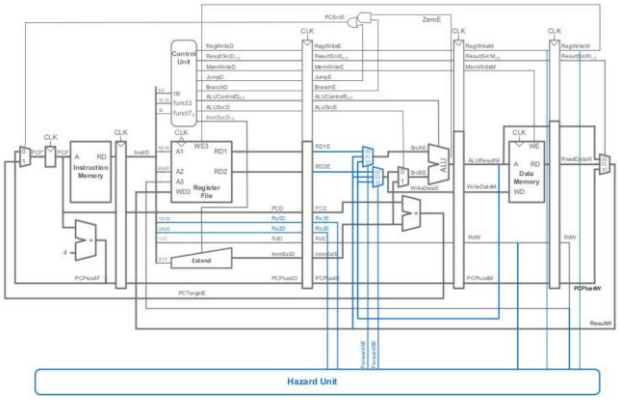

Fig. 23. Updated Pipeline Top Architecture

To resolve RAW hazards dynamically, a dedicated Hazard Unit is integrated into the datapath. Rather than waiting for a computed result to reach the Writeback stage, the Hazard Unit intercepts the resolved data immediately after the ALU (Memory stage) or Data Memory (Writeback stage) and routes it directly back to the Execute stage multiplexers.

Mux control	Source	Explanation
ForwardA = 00	ID/EX	The first ALU operand comes from the register file.
ForwardA = 10	EX/MEM	The first ALU operand is forwarded from the prior ALU result.
ForwardA = 01	MEM/WB	The first ALU operand is forwarded from data memory or an earlier ALU result.
ForwardB = 00	ID/EX	The second ALU operand comes from the register file.
ForwardB = 10	EX/MEM	The second ALU operand is forwarded from the prior ALU result.
ForwardB = 01	MEM/WB	The second ALU operand is forwarded from data memory or an earlier ALU result.

Fig 24. Condition Table

The boolean logic governing this hardware intervention is detailed in Fig. 24. The Hazard Unit continuously compares the destination registers of the older instructions currently residing in the MEM and WB stages (RdM and RdW) against the source registers of the instruction currently residing in the Execute stage (Rs1E and Rs2E).

As shown in the logic diagram, forwarding from the Memory stage (the EX/MEM pipeline register) occurs only if three conditions are strictly met: the previous instruction must actively write to a register (RegWriteM is high), the target register must not be the hardwired zero register (RdM $\neq$ 0), and the destination address must match the current source address. If the dependent data is one cycle older and currently residing in the MEM/WB register, parallel logic evaluates the Writeback stage signals (RegWriteW and RdW).

Memory Stage

if (RegWriteM and (RdM != 0) and (RdM == Rs1E))
ForwardAE = 10

if (RegWriteM and (RdM != 0) and (RdM == Rs2E))
ForwardBE = 10

WriteBack Stage

if (RegWriteW and (RdW != 0) and (RdW == Rs1E))
ForwardAE = 01

if (RegWriteW and (RdW != 0) and (RdW == Rs2E))
ForwardBE = 01

Fig. 25. Condition for Data Hazard

Based on these condition evaluations, the Hazard Unit generates the ForwardAE and ForwardBE multiplexer control signals. The routing behavior dictated by these signals is defined in Fig. 24. A control value of 10_2 overrides the standard ID/EX register file output to select the prior ALU result, while 01_2 selects data forwarded from the MEM/WB stage. If no hazard is detected, the unit defaults to 00_2, permitting standard sequential execution without pipeline stalls.

```
PS D:\Coursera\RISC-V> iverilog -g2012 -o simulation.vvp hazard_unit.v components6.v hazard_unit_tb.sv
PS D:\Coursera\RISC-V> vvp simulation.vvp
--- Starting Pipeline Simulation ---
VCD info: dumpfile simulation.vcd opened for output.
Time: 0 | rst: 1 | RegWM: 0 | RegWW: 0 | Rs1_E:  0 | Rs2_E:  0 | FwdAE: 00 | FwdBE: 00
Time: 2 | rst: 0 | RegWM: 0 | RegWW: 0 | Rs1_E:  0 | Rs2_E:  0 | FwdAE: 00 | FwdBE: 00
Time: 12 | rst: 1 | RegWM: 0 | RegWW: 0 | Rs1_E:  1 | Rs2_E:  2 | FwdAE: 00 | FwdBE: 00
Time: 22 | rst: 1 | RegWM: 1 | RegWW: 0 | Rs1_E: 10 | Rs2_E:  2 | FwdAE: 10 | FwdBE: 00
Time: 32 | rst: 1 | RegWM: 0 | RegWW: 1 | Rs1_E:  1 | Rs2_E: 15 | FwdAE: 00 | FwdBE: 01
Time: 42 | rst: 1 | RegWM: 1 | RegWW: 0 | Rs1_E:  0 | Rs2_E:  0 | FwdAE: 00 | FwdBE: 00
--- Simulation Complete ---
hazard_unit_tb.sv:73: $finish called at 152 (1s)
PS D:\Coursera\RISC-V>
```

Fig. 26. Hazard Unit terminal output (isolated FU test)

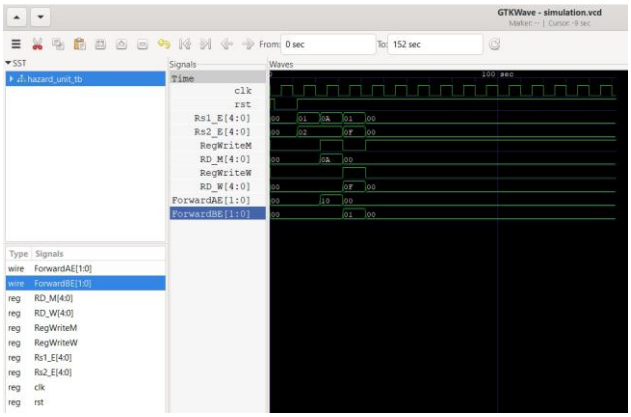

Fig. 27. Hazard Unit GTKWave simulation waveform

To validate the combinatorial logic of the Forwarding Unit, an isolated SystemVerilog testbench was simulated, and the signal transitions were analyzed using GTKWave (Fig. 26 and Fig. 27). The simulation systematically verifies the unit's ability to detect Read-After-Write (RAW) data dependencies across different pipeline stages and properly handle the hardwired zero-register edge case.

At the onset of the simulation, following the release of the active-low reset at t = 12, the datapath simulates a standard execution state with no data hazards. Although source registers are active (Rs1_E = 1, Rs2_E = 2), neither the Memory nor Writeback stages assert a write-enable signal (RegWriteM = 0, RegWriteW = 0). Consequently, the Hazard Unit correctly defaults both multiplexer control signals to 00, permitting standard sequential execution.

At t = 22, the simulation injects an EX/MEM hazard condition. The Memory stage asserts a write operation (RegWriteM = 1) targeting destination register 10 (RD_M = 10). Concurrently, the Execute stage requires register 10 as its first operand (Rs1_E = 10). The waveform

demonstrates that the Hazard Unit instantly detects this immediate dependency, overriding the default behavior and asserting ForwardAE = 10. This dynamically routes the prior ALU result directly into the current execution cycle.

At t = 32, a MEM/WB hazard is simulated for the second ALU operand. The Writeback stage holds valid data designated for register 15 (RegWriteW = 1, RD_W = 15), which matches the source requirement in the Execute stage (Rs2_E = 15). The logic accurately resolves this delayed dependency by asserting ForwardBE = 01, successfully bypassing the stale Register File data in favor of the loaded memory payload.

Finally, at t = 42, the testbench validates a critical architectural edge case regarding the RISC-V hardwired zero register (x0). Despite the Memory stage asserting a write-enable signal intended for destination address 0 (RD_M = 0), and the Execute stage requesting address 0 (Rs1_E = 0), the Hazard Unit correctly identifies that the zero register cannot be overwritten or forwarded. The logic evaluates the (RdM != 0) constraint as false, safely maintaining ForwardAE = 00 and preventing an invalid data bypass.

TOP-LEVEL INTEGRATION

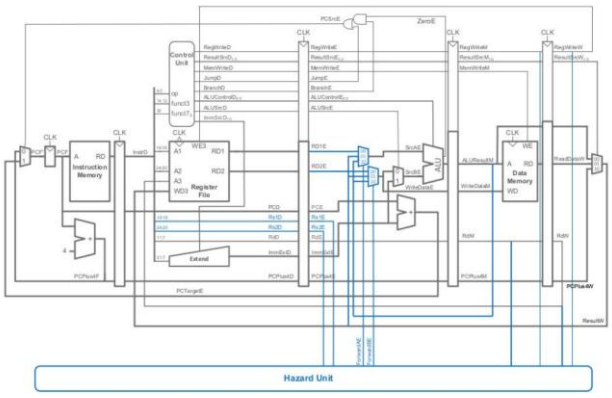

Fig. 28. Pipeline Top Architecture

The final phase of the processor design involves the integration of the five individual pipeline stages—Fetch, Decode, Execute, Memory, and Writeback—into a cohesive top-level architecture (Fig. 28). This datapath is fully augmented by the Hazard Unit, establishing the necessary feedback loops for dynamic data forwarding and branch control to ensure uninterrupted cycle-by-cycle execution.

Fig. 29. Hazard Unit terminal output (top-level test)

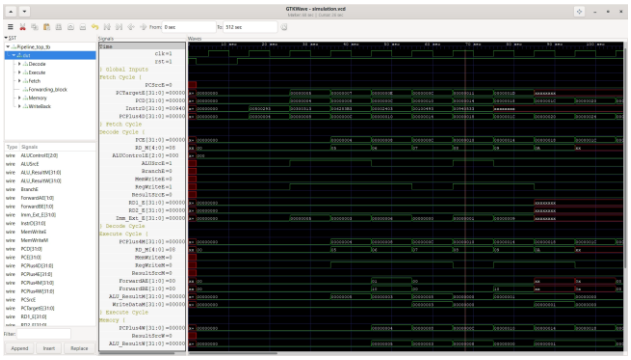

Fig. 30. Hazard Unit GTKWave simulation waveform

To validate the complete, integrated architecture, a custom machine code program was loaded into the Instruction Memory (memfile.hex). This program was explicitly designed to inject immediate values into registers and subsequently force Read-After-Write (RAW) data hazards across consecutive instructions, stress-testing the pipeline's forwarding capabilities under standard execution conditions. The top-level SystemVerilog testbench was simulated, and the signal propagation was monitored across all pipeline boundaries. Fig. 29 summarizes the critical execution cycles, while Fig. 30 visually details the signal transitions.

The simulation initiates with an active-low reset, flushing all uninitialized data (x) from the pipeline registers to guarantee a safe 0x00000000 starting state. Following the release of the reset, the program counter (PC_Dec)

sequentially increments by 4 bytes per clock cycle, successfully fetching the machine code instructions.

- The system's ability to seamlessly execute code and dynamically resolve dependencies is definitively proven between t = 15 and t = 55
- At t = 15 and t = 25, two independent I-type instructions enter the pipeline (0x00500293 and 0x00300313), loading the values 5 and 3 into their respective destination registers.
- At t = 35, an R-type addition instruction (0x006283b3) is fetched, which fundamentally depends on the results of the previous two instructions to calculate its sum.
- At t = 45, this addition instruction reaches the Execute stage (PC_Ex = 0x00000008). Because the required operands have not yet been committed to the Register File, a structural hazard exists. However, the terminal log explicitly confirms the Hazard Unit's successful intervention: the multiplexer control signals assert FwdAE/BE = 01/10. This proves the hardware successfully bypasses stale register data, dynamically routing the value 5 from the MEM/WB boundary (01) and the value 3 from the EX/MEM boundary (10) directly into the ALU.
- Consequently, at t = 55, the ALU correctly computes the sum, outputting 0x00000008 to the ALU_OutM bus, validating the forwarding logic without requiring a single pipeline stall.

As the simulation progresses to conclusion, the terminal log demonstrates the synchronized flow of data reaching the end of the datapath, with the computed values (5, 3, and 8) successfully appearing on the WB_Res bus to be safely committed to the Register File. Unused pipeline slots naturally propagate 0x00000000 or undefined x states until the program fully clears the datapath, confirming the robust, high-throughput functionality of the integrated RISC-V architecture.

CONCLUSION

This project successfully demonstrates the design, implementation, and rigorous verification of a 32-bit, 5-stage pipelined RISC-V (RV32I) processor core. By decomposing the single-cycle datapath into discrete Fetch, Decode, Execute, Memory, and Writeback stages, the architecture significantly increases clock frequency and overall instruction throughput.

The primary technical achievement of this implementation is the successful integration of a dynamic Hardware Hazard Unit. As proven through extensive SystemVerilog testbench simulations, the processor is capable of detecting structural Read-After-Write (RAW) data dependencies in real-time. By utilizing internal data forwarding paths, the architecture dynamically routes computed operands from the EX/MEM and MEM/WB pipeline boundaries directly back to the ALU. The simulation results confirm that this logic entirely eliminates the need for software-inserted NOPs or hardware-induced pipeline stalls during standard arithmetic dependencies, successfully maintaining the theoretical ideal throughput of one instruction per clock cycle (CPI $\approx$ 1.0). Furthermore, the control flow logic successfully resolves branching scenarios by accurately calculating target addresses and flushing stale pipeline data.

Ultimately, this processor design serves as a comprehensive validation of advanced computer architecture principles, showcasing robust RTL design practices, precise combinatorial and sequential logic integration, and thorough functional verification methodologies.

REFERENCES

[1] D. A. Patterson and J. L. Hennessy, Computer Organization and Design RISC-V Edition: The Hardware Software Interface, 2nd ed. Cambridge, MA, USA: Morgan Kaufmann, 2020.

[2] A. Waterman, K. Asanović, and SiFive Inc., The RISC-V Instruction Set Manual, Volume I: User-Level ISA, Document Version 2.2. Berkeley, CA, USA: RISC-V Foundation, 2017.

[3] J. L. Hennessy and D. A. Patterson, Computer Architecture: A Quantitative Approach, 6th ed. Cambridge, MA, USA: Morgan Kaufmann, 2017.

[4] S. Sutherland, S. Davidmann, and P. Flake, SystemVerilog for Design: A Guide to Using SystemVerilog for Hardware Design and Modeling, 2nd ed. New York, NY, USA: Springer, 2006.